

BUSINESS REQUIREMENTS DOCUMENT

Invoice Portal

SaaS-Based Invoice Management System

Project Name	Invoice Portal
Developer	Nehal Khan
Version	1.0
Date	June 2026
Tech Stack	Laravel 11, PHP 8.3, MySQL, Tailwind CSS
Live URL	web-production-15ad2e.up.railway.app
GitHub	github.com/nehaal-dev/invoice_portal

1. Executive Summary

Invoice Portal is a SaaS-based Invoice Management System built using Laravel 11 and PHP 8.3. It allows business owners to manage their clients, create professional invoices, generate PDFs, accept online payments via Stripe, and provide clients with a dedicated portal to view their invoices.

The system is deployed on Railway.app with a MySQL database and supports both a web interface for business owners and a REST API for potential mobile/third-party integrations.

2. Project Scope

2.1 In Scope

- User Authentication (Register, Login, Logout)
- Client Management (CRUD)
- Invoice Management (CRUD) with line items
- PDF Invoice Generation and Download
- Online Payment via Stripe (Test Mode)
- Email Notifications to clients with PDF attachment
- Dashboard with business statistics
- REST API with Sanctum authentication
- Subscription Plans (Free / Pro)
- Client Portal (clients view their own invoices)
- Deployment on Railway.app

2.2 Out of Scope

- Multi-currency support
- Recurring invoice automation
- Mobile application (Android/iOS)
- Multi-user team management
- Accounting / GST filing integration

3. Stakeholders

Stakeholder	Role	Description
Business Owner	Primary User	Creates clients, invoices, manages payments, views dashboard
Client	Secondary User	Receives invoice emails, views invoices via client portal
Developer	Builder	Nehal Khan — builds and maintains the system
Third-party API User	External	Uses REST API to fetch invoice/client data

4. Functional Requirements

4.1 Authentication

ID	Requirement	Priority	Status
FR-01	User can register with name, email, password	High	Complete
FR-02	User can login with email and password	High	Complete
FR-03	User can logout	High	Complete
FR-04	Unauthenticated users are redirected to login	High	Complete
FR-05	Client login redirects to client portal	High	Complete
FR-06	Owner login redirects to dashboard	High	Complete

4.2 Client Management

ID	Requirement	Priority	Status
FR-07	Owner can add a new client (name, email, phone, address, city, country)	High	Complete
FR-08	Owner can view list of their clients only	High	Complete
FR-09	Owner can edit client details	High	Complete
FR-10	Owner can delete a client	High	Complete
FR-11	System auto-creates portal user when client is added	Medium	Complete

4.3 Invoice Management

ID	Requirement	Priority	Status
FR-12	Owner can create invoice with multiple line items	High	Complete
FR-13	System auto-generates invoice number (INV-001, INV-002)	High	Complete
FR-14	System calculates subtotal, tax %, discount (flat), and total	High	Complete

ID	Requirement	Priority	Status
FR-15	Invoice status: Draft, Unpaid, Paid, Overdue	High	Complete
FR-16	Owner can edit invoice and line items	High	Complete
FR-17	Owner can delete invoice	High	Complete
FR-18	Owner can view invoice details	High	Complete
FR-19	Free plan users limited to 5 invoices/month	Medium	Complete

4.4 PDF Generation

ID	Requirement	Priority	Status
FR-20	Owner can download invoice as PDF	High	Complete
FR-21	PDF includes invoice details, line items, totals, client info	High	Complete
FR-22	PDF is auto-attached to invoice email	High	Complete

4.5 Payment System

ID	Requirement	Priority	Status
FR-23	Owner can initiate Stripe payment for unpaid invoices	High	Complete
FR-24	Stripe test card 4242 4242 4242 4242 supported	High	Complete
FR-25	On successful payment, invoice status updates to Paid	High	Complete
FR-26	Payment record saved (amount, date, method, transaction ID)	High	Complete
FR-27	Duplicate payment prevention via transaction ID check	High	Complete
FR-28	Payment confirmation email sent after successful payment	Medium	Complete

4.6 Email Notifications

ID	Requirement	Priority	Status
FR-29	Owner can manually send invoice email to client	High	Complete
FR-30	Email includes invoice details and PDF attachment	High	Complete
FR-31	Email includes Pay Now button for unpaid invoices	Medium	Complete
FR-32	Email includes portal login link and credentials	Medium	Complete
FR-33	Payment confirmation email sent via Gmail SMTP	Medium	Complete

4.7 Dashboard

ID	Requirement	Priority	Status
FR-34	Dashboard shows Total Invoices count	High	Complete
FR-35	Dashboard shows Total Clients count	High	Complete
FR-36	Dashboard shows Total Revenue (sum of paid invoices)	High	Complete
FR-37	Dashboard shows Pending Invoices count	High	Complete

4.8 REST API

ID	Requirement	Priority	Status
FR-38	POST /api/login returns Sanctum Bearer token	High	Complete
FR-39	GET /api/invoices returns authenticated user's invoices	High	Complete
FR-40	GET /api/invoices/{id} returns single invoice	High	Complete
FR-41	POST /api/invoices creates new invoice	High	Complete
FR-42	GET /api/clients returns authenticated user's clients	High	Complete
FR-43	POST /api/clients creates new client	High	Complete

4.9 Subscription Plans

ID	Requirement	Priority	Status
FR-44	Free plan: max 5 invoices per month	High	Complete
FR-45	Pro plan: unlimited invoices (Rs. 999/month)	High	Complete
FR-46	Upgrade via Stripe checkout	High	Complete
FR-47	CheckSubscription middleware enforces limits	High	Complete

4.10 Client Portal

ID	Requirement	Priority	Status
FR-48	Client logs in with email and default password (client123)	High	Complete
FR-49	Client sees only their own invoices	High	Complete
FR-50	Client can view invoice details	High	Complete
FR-51	Client can download PDF of their invoice	Medium	Complete
FR-52	Client cannot access owner dashboard or other client data	High	Complete

5. Non-Functional Requirements

Category	Requirement	Implementation
Security	All routes protected by auth middleware	Laravel Breeze + Sanctum
Security	API authentication via Bearer token	Laravel Sanctum
Security	HTTPS enforced in production	AppServiceProvider::forceScheme
Security	CSRF protection on all forms	Laravel default
Performance	Database queries optimized with Eloquent	where(user_id) on all queries
Scalability	Hosted on Railway.app with MySQL	Cloud deployment
Usability	Responsive design for mobile and desktop	Tailwind CSS responsive classes
Maintainability	MVC architecture	Laravel Controllers, Models, Views
Availability	24/7 uptime on Railway.app	Cloud hosting

6. Database Schema

6.1 Tables Overview

Table	Purpose	Key Columns
users	Business owners and client portal users	id, name, email, password, role, client_id
clients	Business owner's clients	id, user_id, client_name, email, phone, address, city, country
invoices	Invoice records	id, user_id, client_id, invoice_number, status, subtotal, tax, discount, total
items	Invoice line items	id, invoice_id, item_name, quantity, price, total
payments	Payment records	id, invoice_id, amount, payment_date, payment_method, transaction_id
subscriptions	User subscription plans	id, user_id, plan_name, status, amount, starts_at, expires_at
personal_access_tokens	Sanctum API tokens	id, tokenable_type, tokenable_id, name, token

6.2 Key Relationships

- User hasMany Clients, Invoices | hasOne Subscription
- Client belongsTo User | hasMany Invoices
- Invoice belongsTo User, Client | hasMany Items, Payments
- InvoiceItem belongsTo Invoice (table name: items)
- Payment belongsTo Invoice
- Subscription belongsTo User

7. REST API Documentation

7.1 Authentication

All protected endpoints require Bearer Token in Authorization header.

Method	Endpoint	Description	Auth Required
POST	/api/login	Get Bearer token	No

7.2 Invoice Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/invoices	Get all invoices	Yes (Bearer)
GET	/api/invoices/{id}	Get single invoice	Yes (Bearer)
POST	/api/invoices	Create invoice	Yes (Bearer)

7.3 Client Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/clients	Get all clients	Yes (Bearer)
POST	/api/clients	Create client	Yes (Bearer)

7.4 Sample API Response

GET /api/invoices — Sample JSON Response:

```
[{ "id": 1, "invoice_number": "INV-001", "status": "paid", "total": "25000.00",  
  "client_id": 1 }]
```


8. Technology Stack

Layer	Technology	Purpose
Backend Framework	Laravel 11	MVC framework, routing, ORM, middleware
Language	PHP 8.3	Server-side programming
Database	MySQL	Data storage
Frontend	Blade Templates + Tailwind CSS	Server-side rendering, responsive UI
Authentication	Laravel Breeze + Sanctum	Web auth (session) + API auth (token)
PDF Generation	barryvdh/laravel-dompdf	Generate invoice PDFs
Payment Gateway	Stripe (Test Mode)	Online invoice payments
Email	Gmail SMTP	Invoice and payment confirmation emails
Deployment	Railway.app	Cloud hosting with GitHub auto-deploy
Version Control	Git + GitHub	Source code management
API Testing	Postman	REST API testing

9. System Architecture

9.1 Request Flow

- User sends HTTP request from browser or API client
- Laravel routes.php matches the URL pattern
- Middleware runs (auth check, subscription check, CSRF)
- Controller method executes business logic
- Model interacts with MySQL database via Eloquent ORM
- Controller returns response — Blade view (HTML) or JSON

9.2 Multi-tenancy

Every database query filters by `user_id` to ensure data isolation between business owners. Each owner sees only their own clients, invoices, and subscriptions.

9.3 Deployment Architecture

- Code hosted on GitHub (nehaal-dev/invoice_portal)
- Railway.app connected to GitHub — auto-deploys on git push
- Separate MySQL service on Railway.app
- Environment variables stored in Railway.app dashboard
- Start command: `php artisan migrate --force && php artisan storage:link && php artisan serve`

10. Security Requirements

Security Measure	Implementation
Authentication	Laravel Breeze session-based auth for web
API Authentication	Sanctum Bearer token for REST API
CSRF Protection	Laravel CSRF tokens on all forms
Password Hashing	bcrypt hashing via Laravel
HTTPS Enforcement	URL::forceScheme('https') in AppServiceProvider
Data Isolation	user_id filter on all queries
Role-based Access	owner vs client role with middleware
Payment Security	Stripe handles card data — never stored locally

11. Future Enhancements

Feature	Description	Priority
AI Description Generator	OpenAI API to auto-generate item descriptions	Medium
Search & Filter	Search invoices by client, status, date range	High
Pagination	Paginate invoice and client lists	High
Recurring Invoices	Auto-generate invoices on schedule	Low
Multi-currency	Support USD, EUR, GBP etc.	Low
Invoice Templates	Multiple PDF design templates	Medium
Mobile App	Android/iOS app using REST API	Medium
Password Reset for Clients	Email-based password reset link	High

12. Glossary

Term	Definition
BRD	Business Requirements Document — describes what the system should do
SaaS	Software as a Service — cloud-based software accessed via browser
MVC	Model-View-Controller — architectural pattern used in Laravel
ORM	Object-Relational Mapper — Eloquent maps DB tables to PHP classes
CRUD	Create, Read, Update, Delete — basic database operations
REST API	Representational State Transfer API — standard for web services
Sanctum	Laravel package for API token authentication
Stripe	Payment gateway used for online invoice payments
DomPDF	PHP library for generating PDF files from HTML
Railway.app	Cloud platform used for deploying the application
Middleware	Code that runs between request and controller — used for auth checks
Bearer Token	API authentication token sent in Authorization header
Migration	Laravel files that define and modify database table structure
Route Model Binding	Laravel automatically finds Model by ID from URL parameter